```
"""
SLE-2 Profiling: BFS vs DFS on a maze (grid
graph)
Course: 02AML204 - Introduction to
Artificial Intelligence

We build a 20x20 grid maze with some walls,
represented as a graph,
and compare BFS vs DFS for finding a path
from start to goal.

Metrics collected:
- Execution time (ms), averaged over
multiple runs
- Nodes expanded (states explored)
- Path length found
"""

import time
import random
from collections import deque

random.seed(42)

# ----------------------------------------
----------------
# 1. Build the maze (20x20 grid, ~20%
walls)
# ----------------------------------------
----------------
```

```
ROWS, COLS = 20, 20
WALL_PROB = 0.20

def build_maze(rows, cols, wall_prob):
    maze = [[0] * cols for _ in
range(rows)]
    for r in range(rows):
        for c in range(cols):
            if random.random() < wall_prob:
                maze[r][c] = 1  # wall
    maze[0][0] = 0
    maze[rows - 1][cols - 1] = 0
    return maze

maze = build_maze(ROWS, COLS, WALL_PROB)
START = (0, 0)
GOAL = (ROWS - 1, COLS - 1)

def neighbors(node):
    r, c = node
    for dr, dc in [(-1, 0), (1, 0), (0,
-1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc <
COLS and maze[nr][nc] == 0:
            yield (nr, nc)

# ----------------------------------------
----------------
# 2. BFS implementation
# ----------------------------------------
----------------
def bfs(start, goal):
    frontier = deque([start])
```

```
    visited = {start}
    parent = {start: None}
    nodes_expanded = 0

    while frontier:
        node = frontier.popleft()
        nodes_expanded += 1
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1],
nodes_expanded

        for nxt in neighbors(node):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = node
                frontier.append(nxt)

    return None, nodes_expanded

# ----------------------------------------
----------------
# 3. DFS implementation (iterative, with
visited set)
# ----------------------------------------
----------------
def dfs(start, goal):
    frontier = [start]
    visited = {start}
    parent = {start: None}
    nodes_expanded = 0
```

```
    while frontier:
        node = frontier.pop()
        nodes_expanded += 1
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1],
nodes_expanded

        for nxt in neighbors(node):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = node
                frontier.append(nxt)

    return None, nodes_expanded

# ----------------------------------------
----------------
# 4. Run each algorithm 3 times, record
time + nodes expanded
# ----------------------------------------
----------------
def run_trials(algo_fn, start, goal,
trials=3):
    times = []
    nodes_list = []
    path_len = None
    for _ in range(trials):
        t0 = time.perf_counter()
        path, nodes_expanded =
```

```
algo_fn(start, goal)
        t1 = time.perf_counter()
        times.append((t1 - t0) * 1000)  #
ms
        nodes_list.append(nodes_expanded)
        if path is not None:
            path_len = len(path)
    return times, nodes_list, path_len

if __name__ == "__main__":
    print(f"Maze size: {ROWS}x{COLS}, wall
probability: {WALL_PROB}")
    print(f"Start: {START}, Goal:
{GOAL}\n")

    bfs_times, bfs_nodes, bfs_path_len =
run_trials(bfs, START, GOAL, trials=3)
    dfs_times, dfs_nodes, dfs_path_len =
run_trials(dfs, START, GOAL, trials=3)

    print("=== BFS ===")
    print("Times (ms):", [round(t, 4) for t
in bfs_times])
    print("Nodes expanded per run:",
bfs_nodes)
    print("Path length found:",
bfs_path_len)
    print("Average time (ms):",
round(sum(bfs_times) / len(bfs_times), 4))
    print("Average nodes expanded:",
round(sum(bfs_nodes) / len(bfs_nodes), 2))

    print("\n=== DFS ===")
    print("Times (ms):", [round(t, 4) for t
```

```
in dfs_times])
    print("Nodes expanded per run:",
dfs_nodes)
    print("Path length found:",
dfs_path_len)
    print("Average time (ms):",
round(sum(dfs_times) / len(dfs_times), 4))
    print("Average nodes expanded:",
round(sum(dfs_nodes) / len(dfs_nodes), 2))

    print("\n=== Summary Table ===")
    print(f"{'Metric':<20}{'BFS':<15}
{'DFS':<15}")
    print(f"{'Avg Time (ms)':<20}
{round(sum(bfs_times)/len(bfs_times),4):
<15}
{round(sum(dfs_times)/len(dfs_times),4):
<15}")
    print(f"{'Nodes Expanded':<20}
{round(sum(bfs_nodes)/len(bfs_nodes),2):
<15}
{round(sum(dfs_nodes)/len(dfs_nodes),2):
<15}")
    print(f"{'Path Length':<20}
{bfs_path_len:<15}{dfs_path_len:<15}")
```